Trees

Chapter 11

- With Question/Answer Animations

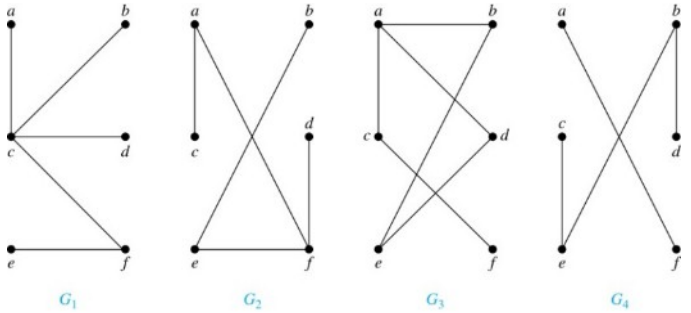
© 2019 McGraw-Hill Education

Introduction to Trees

Section 11.1

Trees 1

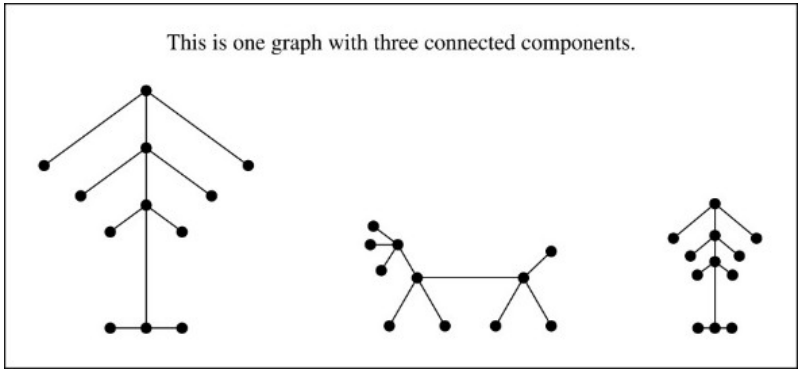
Definition: A *tree* is a connected undirected graph with no simple circuits回路.



Example: Which of these graphs are trees?

Solution: G_1 and G_2 are trees - both are connected and have no simple circuits. Because *e*, *b*, *a*, *d*, *e* is a simple circuit, G_3 is not a tree. G_4 is not a tree because it is not connected.

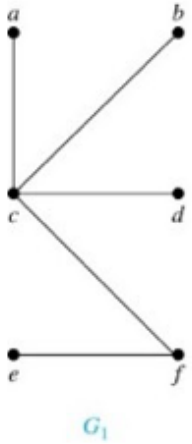
Definition: A *forest* is a graph that has no simple circuit, but is not connected. Each of the connected components in a forest is a tree.



Example of a forest

Jump to long description

Trees 2



G_1

Theorem: An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

Proof: Assume that *T* is a tree. Then *T* is connected with no simple circuits. Hence, if *x* and *y* are distinct vertices of *T*, there is a simple path between them (by Theorem 1 of Section 10.4). This path must be unique - for if there were a second path, there would be a simple circuit in *T* (by Exercise 59 of Section 10.4). Hence, there is a unique simple path between any two vertices of a tree.

Now assume that there is a unique simple path between any two vertices of a graph *T*. Then *T* is connected because there is a path between any two of its vertices. Furthermore, *T* can have no simple circuits since if there were a simple circuit, there would be two paths between some two vertices. Then T is a tree.

Hence, a graph with a unique simple path between any two vertices is a tree.

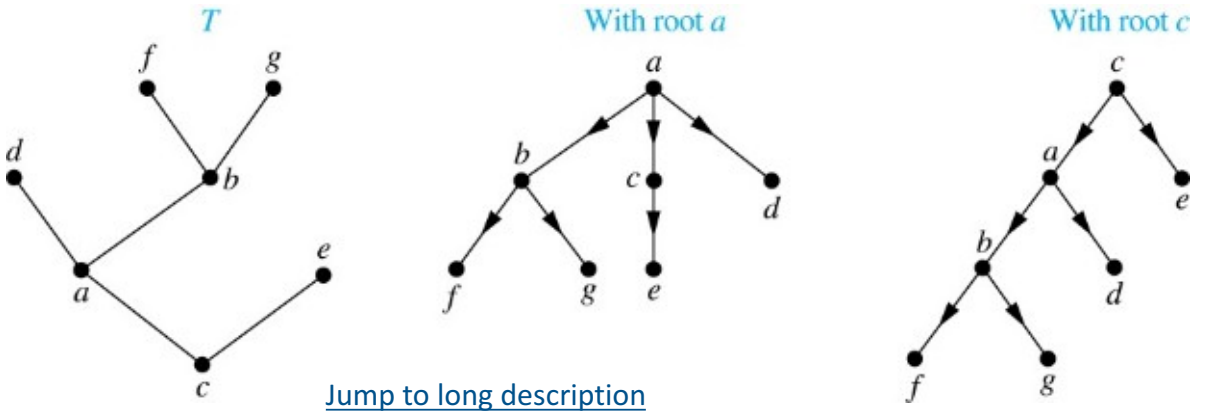
11.1.1 Rooted Trees

Definition: A *rooted tree* (根树)is a tree in which one vertex has been designated as the *root* and every edge is directed away from the root.

An unrooted tree is converted into different rooted trees when different vertices are chosen as the root.

- ***Applications of rooted trees***:

Decision trees, organization charts, tournament competition, database, computer programming (sorting and searching) etc.



Jump to long description

Rooted Tree Terminology

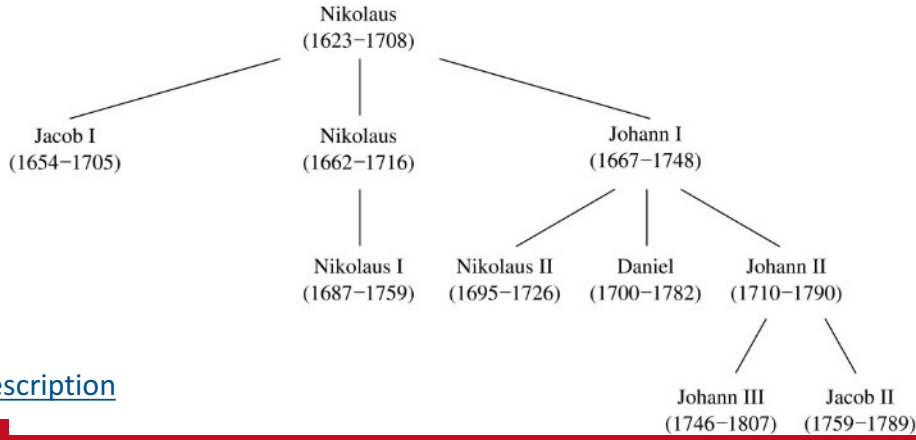
Terminology for rooted trees is a mix from botany 植物学and genealogy 家谱.

If *v* is a vertex of a rooted tree other than the root, the *parent* 父节点of *v* is the unique vertex *u* such that there is a directed edge from *u* to *v*. When *u* is a parent of *v*, *v* is called a *child* 结点of *u*. Vertices with the same parent are called *siblings*.兄弟姐妹

The *ancestors* 祖先of a vertex are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root. The *descendants* 后代of a vertex *v* are those vertices that have *v* as an ancestor.

A vertex of a rooted tree with no children is called a *leaf* 叶. Vertices that have children are called *internal vertices*.内顶点 (root is also an internal vertices)

If *a* is a vertex in a tree, the *subtree* 子树with *a* as its root is the subgraph of the tree consisting of *a* and its descendants and all edges incident to these descendants.



Jump to long description

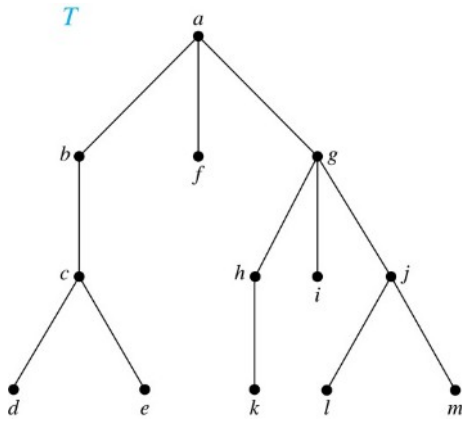
Terminology for Rooted Trees

Example: In the rooted tree *T* (with root *a*):

(i) Find the parent of *c*, the children of *g*, the siblings of *h*, the ancestors of *e*, and the descendants of *b*.

(ii) Find all internal vertices and all leaves.

(iii) What is the subtree rooted at *g*?

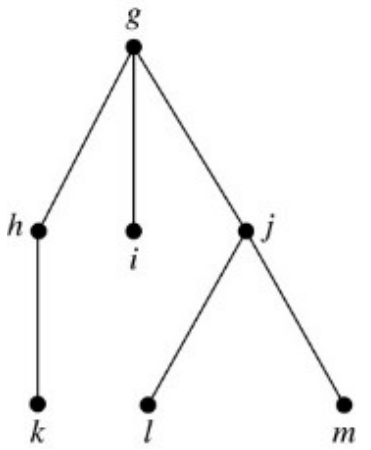


Solution:

(i) The parent of c is b. The children of g are h, i, and j. The siblings of h are i and j. The ancestors of e are c, b, and a. The descendants of b are c, d, and e.

(ii) The internal vertices are a, b, c, g, h, and j. The leaves are d, e, f, i, k, l, and m.

(iii) We display the subtree rooted at g.

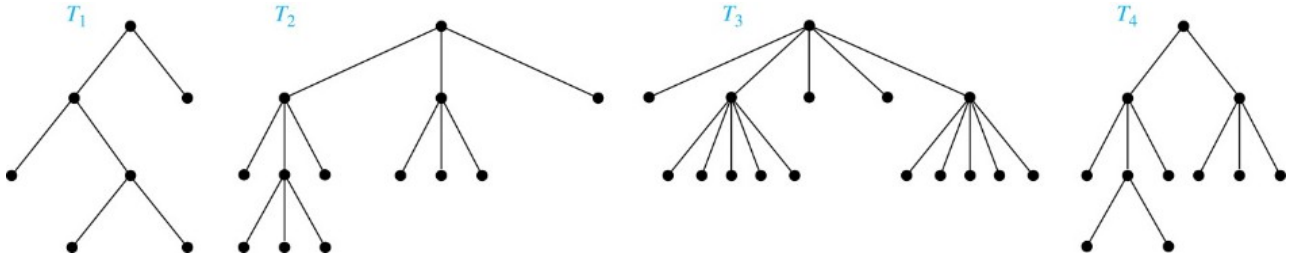


Jump to long description

m-ary Rooted Trees

Definition: A rooted tree is called an *m-ary tree (m元树)* if every internal vertex has no more than *m* children. The tree is called a *full m-ary tree (完全m元树)* if every internal vertex has exactly *m* children. An *m*-ary tree with *m* = 2 is called a *binary* tree.(二元树for binary tree, full is not nessesary)

Example: Are the following rooted trees full *m*-ary trees for some positive integer *m*?



Solution: T1 is a full binary tree because each of its internal vertices has two children. T2 is a full 3-ary tree because each of its internal vertices has three children. In T3 each internal vertex has five children, so T3 is a full 5-ary tree. T4 is not a full m-ary tree for any m because some of its internal vertices have two children and others have three children.

Jump to long description

Ordered Rooted Trees

Definition: An *ordered rooted tree(有序有根树)* is a rooted tree where the children of each internal vertex are ordered.

- We draw ordered rooted trees so that the children of each internal vertex are shown in order from left to right.

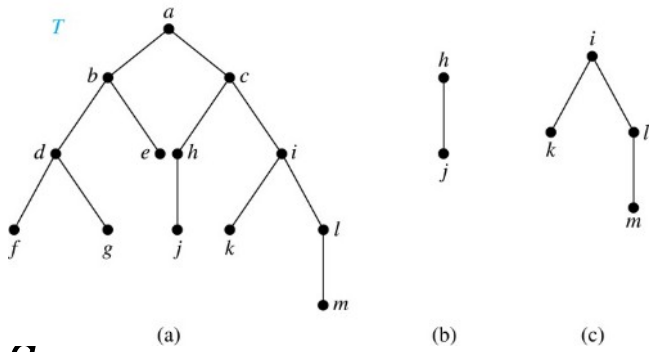
Definition: In an ordered binary tree (usually called just a binary tree), if an internal vertex of a binary tree has two children, the first is called the *left child (左子节点)* and the second the *right child (右子节点)*. The tree rooted at the left child of a vertex is called the *left subtree (左子树)* of this vertex, and the tree rooted at the right child of a vertex is called the *right subtree(右子树)* of this vertex.

Example: Consider the binary tree *T*.

I. What are the left and right children of *d*?

II. What are the left and right subtrees of *c*?

Solution:

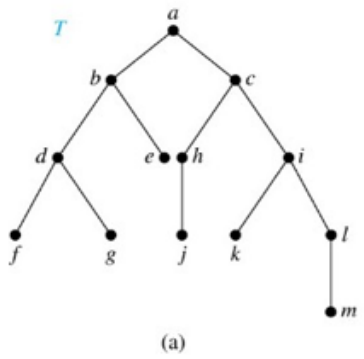


I. The left child of *d* is *f* and the right child is *g*.

II. The left and right subtrees of *c* are displayed in (b) and (c).

Jump to long description

11.1.3 Properties of Trees



Theorem 2: A tree with *n* vertices has *n* − 1 edges.

***Proof* (*by mathematical induction*):**

BASIS STEP: When *n* = 1, a tree with one vertex has no edges. Hence, the theorem holds when *n* = 1.

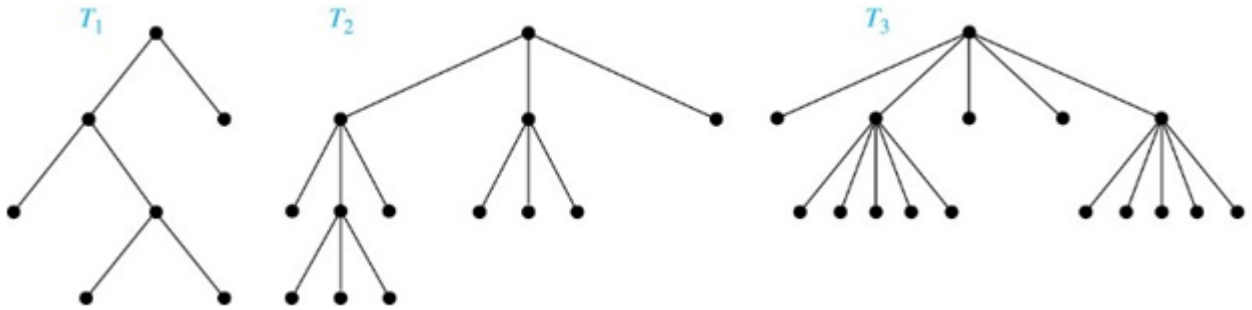
INDUCTIVE STEP: Assume that every tree with *k* vertices has *k* − 1 edges.

Suppose that a tree *T* has *k* + 1 vertices and that *v* is a leaf of *T*. Let *w* be the parent of *v*. Removing the vertex *v* and the edge connecting *w* to *v* produces a tree *T*′ with *k* vertices. By the inductive hypothesis, *T*′ has *k* − 1 edges. Because *T* has one more edge than *T*′, we see that *T* has *k* edges. This completes the inductive step.

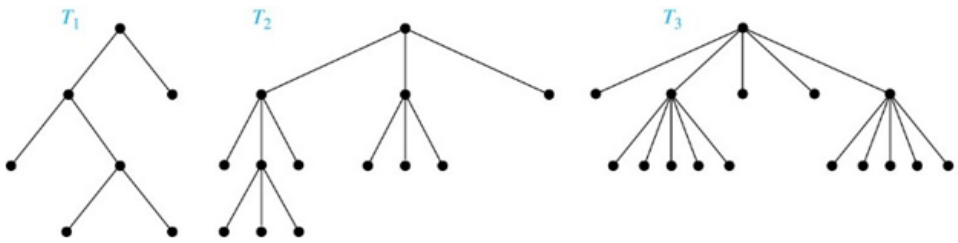
Counting Vertices in Full *m*-Ary Trees[1]

Theorem 3: A full *m*-ary tree with *i* internal vertices has $n = mi + 1$ vertices.

Proof : Every vertex, except the root, is the child of an internal vertex. Because each of the *i* internal vertices has *m* children, there are *mi* vertices in the tree other than the root. Hence, the tree contains $n = mi + 1$ vertices.



Counting Vertices in Full *m*-Ary Trees 2



Theorem 4: A full *m*-ary tree with

I. *n* vertices has $i = (n - 1)/m$ internal vertices and $l = [(m - 1)n + 1]/m$ leaves,

II. *i* internal vertices has $n = mi + 1$ vertices and $l = (m - 1)i + 1$ leaves,

III. *l* leaves has $n = (ml - 1)/(m - 1)$ vertices and $i = (l - 1)/(m - 1)$ internal vertices.

Proof (of part i): Solving for *i* in $n = mi + 1$ (from Theorem 3) gives $i = (n - 1)/m$. Since each vertex is either a leaf or an internal vertex, $n = l + i$. By solving for *l* and using the formula for *i*, we see that

$$l = n - i = n - (n - 1)/m = [(m - 1)n + 1]/m.$$

Level of vertices and height of trees

When working with trees, we often want to have rooted trees where the subtrees at each vertex contain paths of approximately the same length.

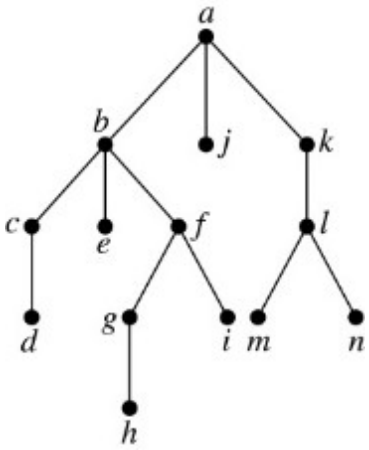
To make this idea precise we need some definitions:

- The *level* 层of a vertex *v* in a rooted tree is the length of the unique path from the root to this vertex.
- The *height* 高度of a rooted tree is the maximum of the levels of the vertices.

Example:

I. Find the level of each vertex in the tree to the right.

II. What is the height of the tree?



Solution:

I. The root *a* is at level 0. Vertices *b*, *j*, and *k* are at level 1. Vertices *c*, *e*, *f*, and *l* are at level 2. Vertices *d*, *g*, *i*, *m*, and *n* are at level 3. Vertex *h* is at level 4.

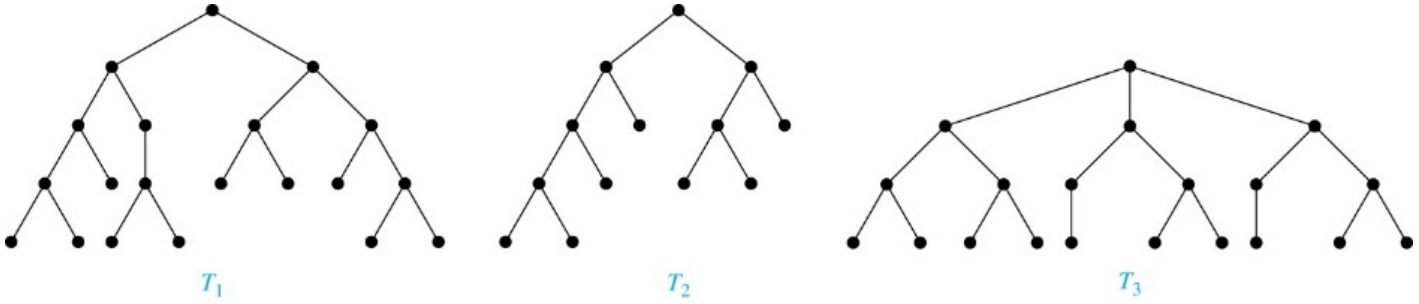
II. The height is 4, since 4 is the largest level of any vertex.

Jump to long description

Balanced *m*-Ary Trees

Definition: A rooted *m*-ary tree of height *h* is *balanced* 均衡if all leaves are at levels *h* or *h* – 1.

Example: Which of the rooted trees shown below is balanced?

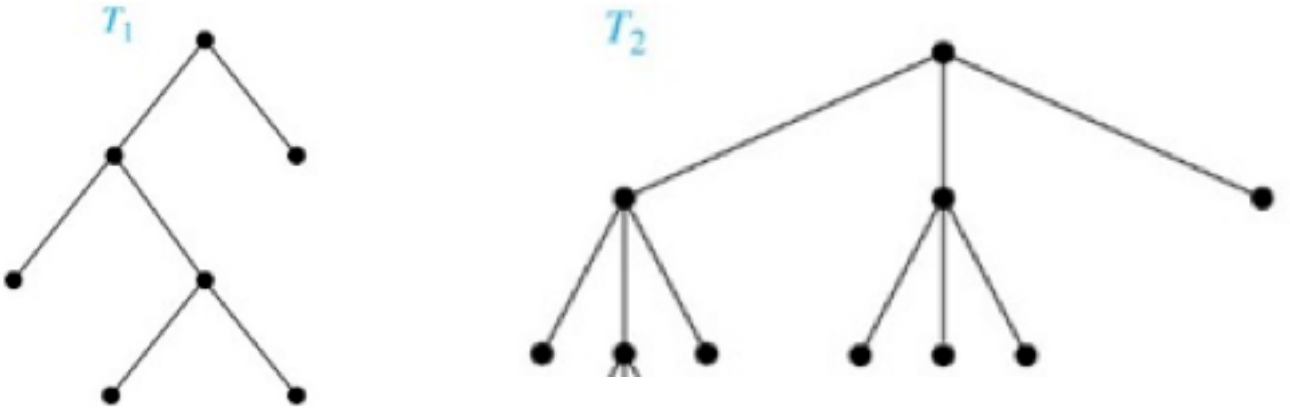


Solution: T_1 and T_3 are balanced, but T_2 is not because it has leaves at levels 2, 3, and 4.

Jump to long description

The Bound for the Number of Leaves in an *m*-Ary Tree

Theorem 5: There are at most m^h leaves in an *m*-ary tree of height *h*. (*see text for the proof*)



Corollary 1: If an m-ary tree of height h has l leaves, then $h \geq \lceil \log_m l \rceil$. If the m-ary tree is full and balanced, then $h = \lceil \log_m l \rceil$. (We are using the ceiling function here. Recall that $\lceil x \rceil$ is the smallest integer greater than or equal to x)

Proof: We know that $l \leq m^h$ from Theorem 5. Taking logarithms to the base m shows that $\log_m l \leq h$. Because h is an integer, we have $h \geq \lceil \log_m l \rceil$. Now suppose that the tree is balanced. Then each leaf is at level h or $h - 1$, and because the height is h, there is at least one leaf at level h. It follows that there must be more than m^{h-1} leaves (see Exercise 30). Because $l \leq m^h$, we have $m^{h-1} < l \leq m^h$. Taking logarithms to the base m in this inequality gives $h - 1 < \log_m l \leq h$. Hence, $h = \lceil \log_m l \rceil$. ◁

Section Summary 1

1.Introduction to Trees: *tree, forest.*

2.Rooted Trees : *parent, child* 结点, *siblings*兄弟姐妹, *ancestors* 祖先, *descendants* 后代, *leaf* 叶, *internal vertices*内顶点, *subtree* 子树.

A rooted tree is called an *m-ary tree* (*m*元树)if every internal vertex has no more than *m* children. The tree is called a *full m-ary tree* (完全*m*元树)if every internal vertex has exactly *m* children. An *m*-ary tree with *m* = 2 is called a *binary* tree.(二元树)

ordered rooted tree(有序有根树) *binary tree : left child* (左子节点) *right child* (右子节点). *left subtree* (左子树) *right subtree*(右子树) .

3.Properties of Trees

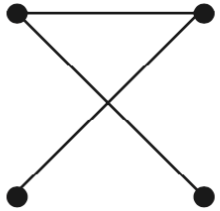
A tree with *n* vertices has $n - 1$ edges.

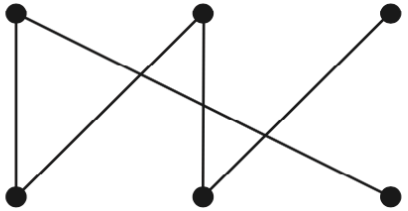
A full *m*-ary tree with *i* internal vertices has $n = mi + 1$ vertices.

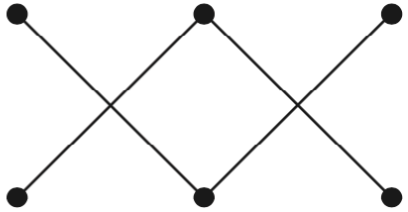
Level of vertices and height of trees. Balanced *m*-Ary Trees.

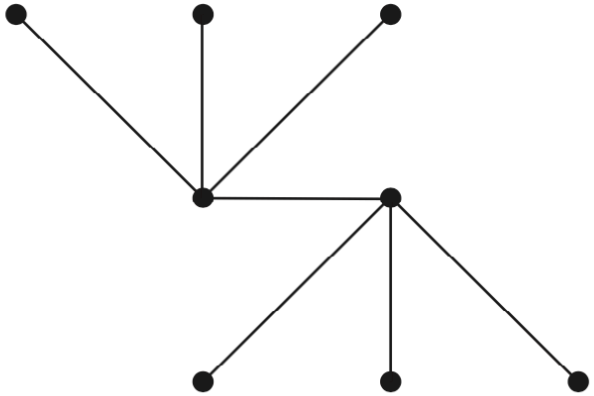
There are at most m^h leaves in an *m*-ary tree of height *h*.

2. Which of these graphs are trees?

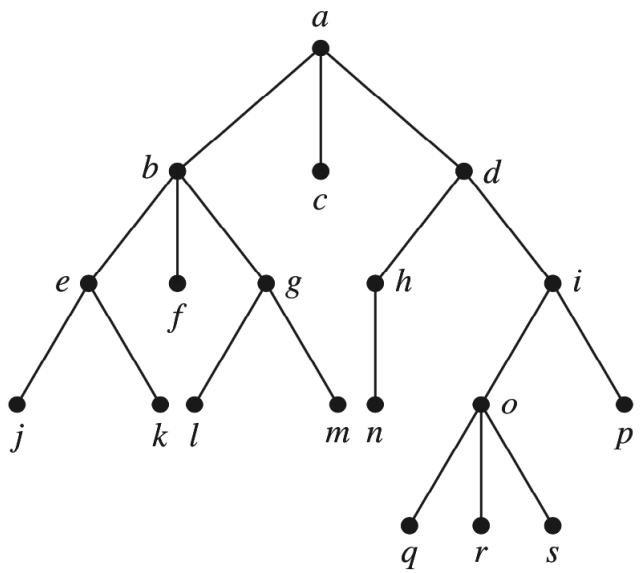
a) 

b) 

c) 

d) 

4. Answer the same questions as listed in Exercise 3 for the rooted tree illustrated.



a) Which vertex is the root?

b) Which vertices are internal?

c) Which vertices are leaves?

d) Which vertices are children of j?

e) Which vertex is the parent of h?

f) Which vertices are siblings of o?

g) Which vertices are ancestors of m?

h) Which vertices are descendants of b?

6. Is the rooted tree in Exercise 4 a full m-ary tree for some positive integer m?

8. What is the level of each vertex of the rooted tree in Exercise 4?

10. Draw the subtree of the tree in Exercise 4 that is rooted at

a) a. **b)** c. **c)** e.

19. How many edges does a full binary tree with 1000 internal vertices have?

20. How many leaves does a full 3-ary tree with 100 vertices have?

Exercise

Page 791 1bcd 3 5 7 9b 17 18

Tree Traversal

Section 11.3

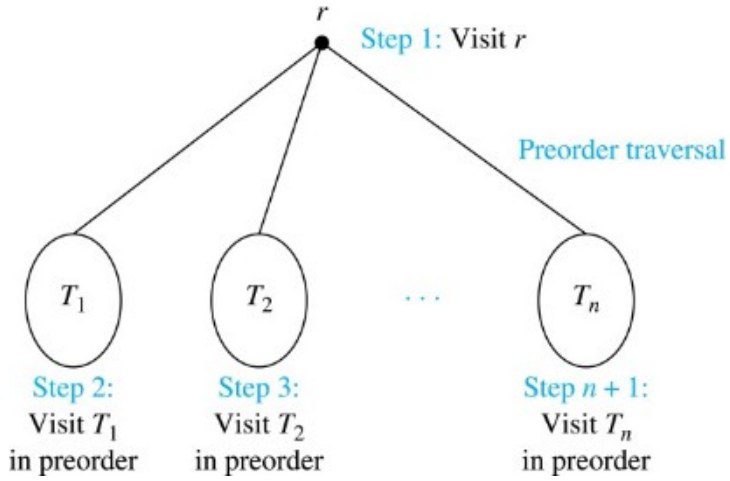
Tree Traversal (树遍历)

Procedures for systematically visiting every vertex of an ordered tree are called *traversals*.

The three most commonly used *traversals* are *preorder traversal*前序遍历, *inorder traversal*中序遍历, and *postorder traversal*后序遍历.

Preorder Traversal前序遍历1

Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *preorder traversal of* T. Otherwise, suppose that T_1, T_2, …, T_n are the subtrees of r from left to right in T. The preorder traversal begins by visiting r, and continues by traversing T_1 in preorder, then T_2 in preorder, and so on, until T_n is traversed in preorder.



Jump to long description

Preorder Traversal 2

procedure *preorder* (*T*: ordered rooted tree)

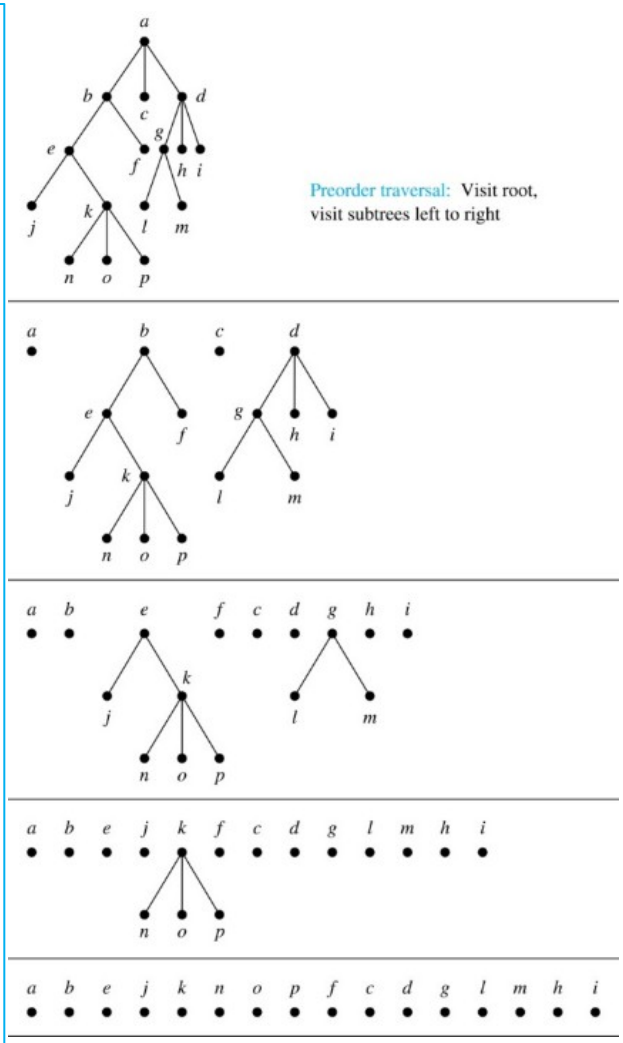
r := root of *T*

list *r*

for each child *c* of *r* from left to right

T(*c*) := subtree with *c* as root

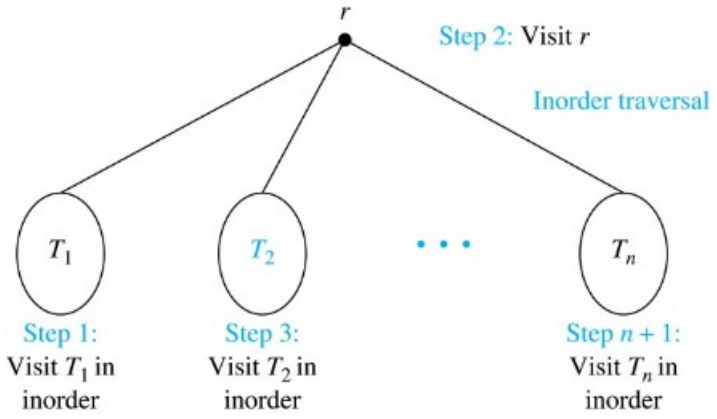
preorder(*T*(*c*))



Jump to long description

Inorder Traversal 中序遍历1

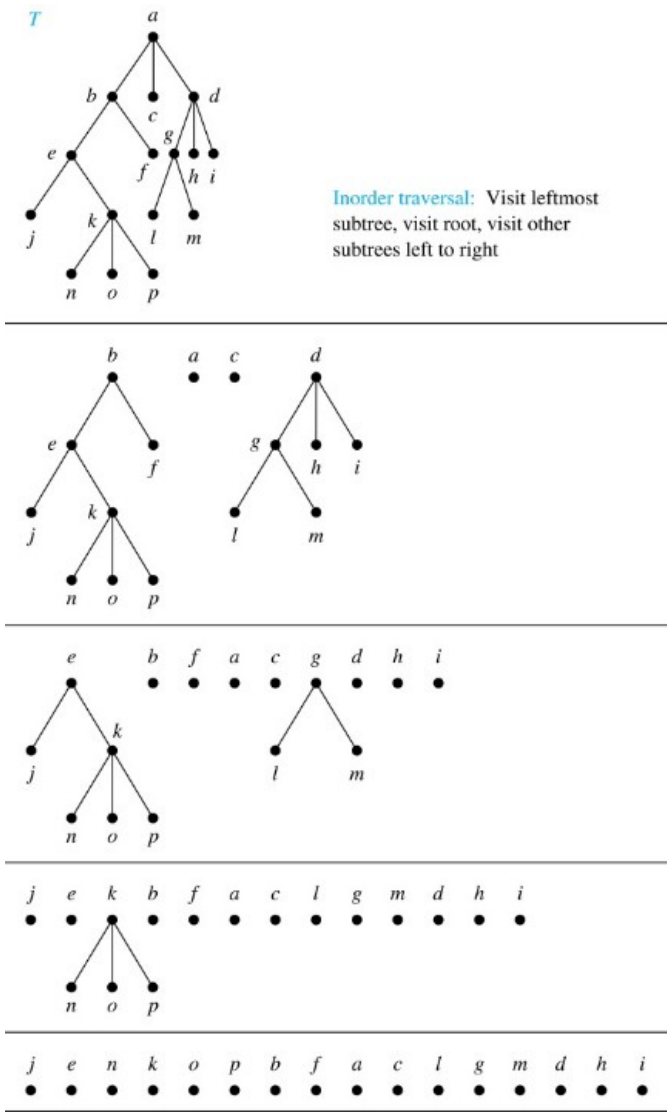
Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *inorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees of r from left to right in T. The inorder traversal begins by traversing T_1 in inorder, then visiting r, and continues by traversing T_2 in inorder, and so on, until T_n is traversed in inorder.



Jump to long description

Inorder Traversal 2

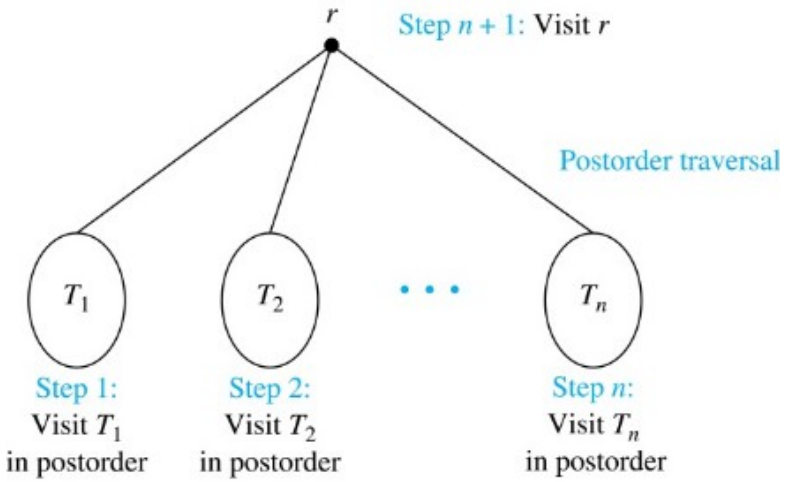
procedure *inorder* (*T*: ordered rooted tree)
r := root of *T*
if *r* is a leaf **then** list *r*
else
  l := first child of *r* from left to right
  T(*l*) := subtree with *l* as its root
  inorder(*T*(*l*))
  list(*r*)
  for each child *c* of *r* except for *l* from left to right
    T(*c*) := subtree with *c* as root
    inorder(*T*(*c*))



Jump to long description

Postorder Traversal后序遍历1后序遍历

Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *postorder traversal of T*. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees of r from left to right in T. The postorder traversal begins by traversing T_1 in postorder, then T_2 in postorder, and so on, after T_n is traversed in postorder, r is visited.



Jump to long description

Postorder Traversal 2

procedure *postorder* (*T*: ordered rooted tree)

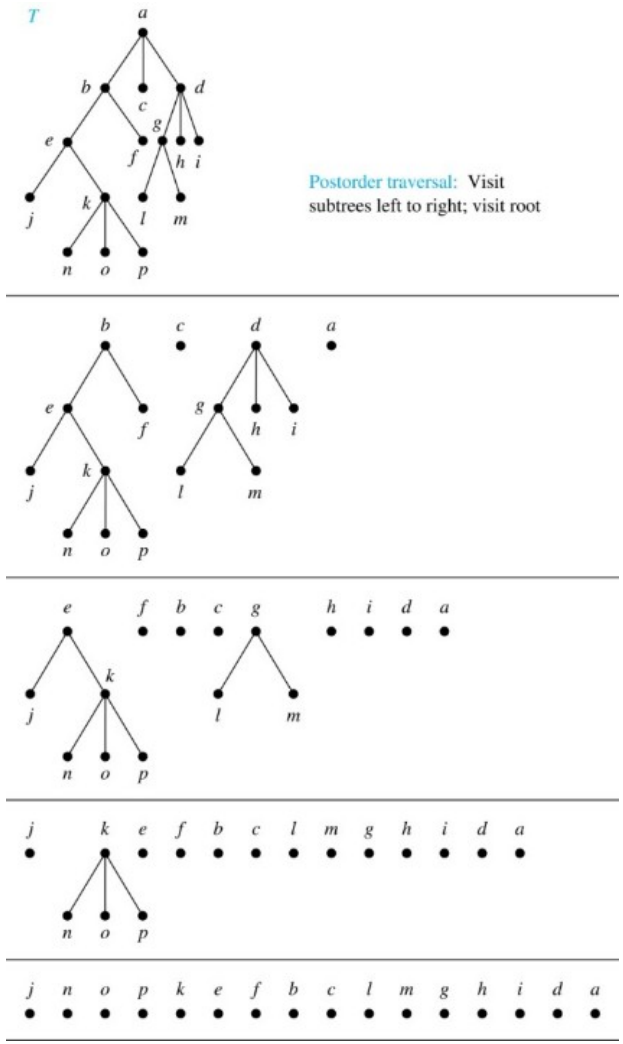
r := root of *T*

for each child *c* of *r* from left to right

T(*c*) := subtree with *c* as root

postorder(*T*(*c*))

list *r*



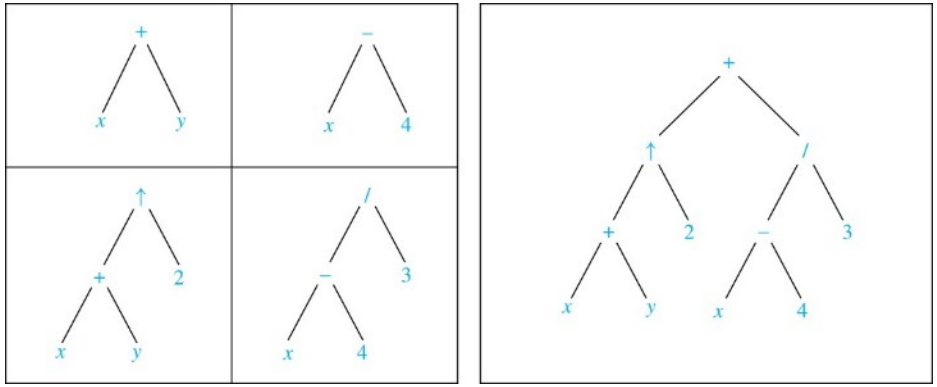
Jump to long description

11.3.4 Expression Trees

Complex expressions can be represented using ordered rooted trees有序有根树.

Consider the expression $\big((x+y)\uparrow 2\big)+\big((x-4)/3\big)$.

A binary tree for the expression can be built from the bottom up, as is illustrated here.



Jump to long description

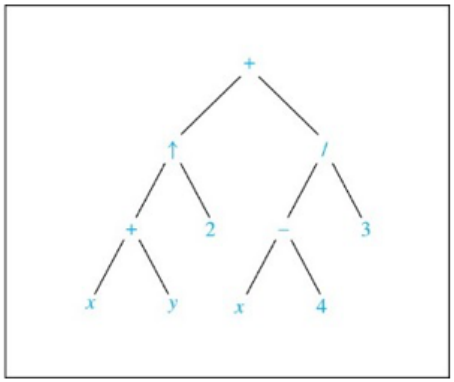
Infix Notation中缀表示法

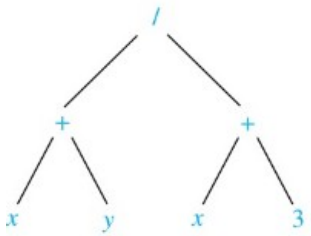
An inorder traversal of the binary tree representing an expression produces the original expression when parentheses are included except for unary operations一元运算, which instead immediately follow their operands运算数.

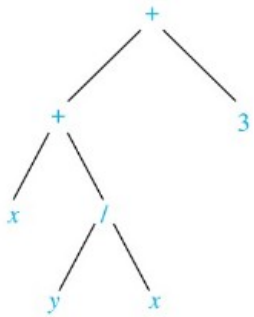
We illustrate why parentheses括弧 are needed with an example that displays three trees all yield the same infix representation.

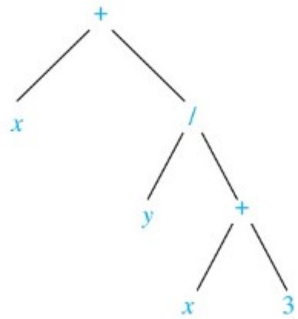
For instance, in- order traversals of the binary trees in Figure, which represent the expressions $(x + y)/(x + 3)$, $(x + (y/x)) + 3$, and $x + (y/(x + 3))$, all lead to the infix expression 中缀表达式$x + y/x + 3$.

$$\big((x+y)\uparrow 2\big)+\big((x-4)/3\big).$$



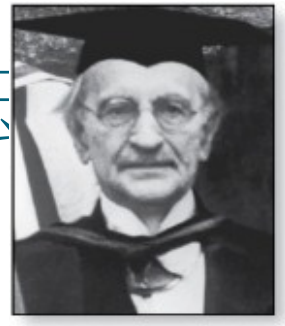






Jump to long description

Prefix Notation 前缀表示法

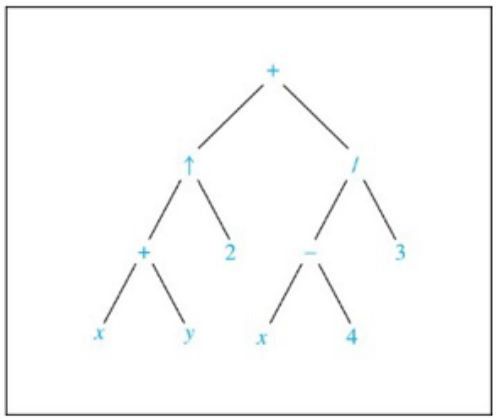


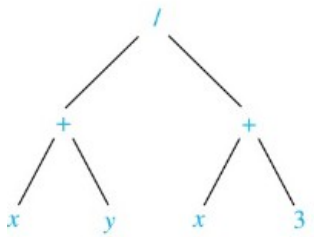
Jan Łukasiewicz
(1878-1956)

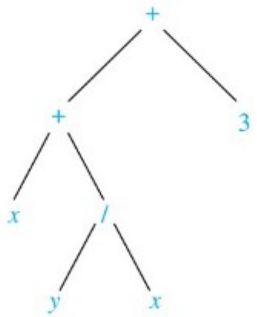
When we traverse the rooted tree representation of an expression in preorder, we obtain the *prefix form* of the expression. Expressions in prefix form are said to be in *Polish notation*波兰符号法, named after the Polish logician Jan Łukasiewicz.

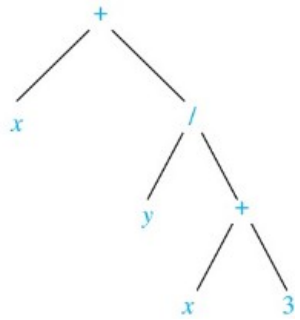
Operators precede their operands in the prefix form of an expression. Parentheses are not needed as the representation is unambiguous.

The prefix form of $((x + y) \uparrow 2) + ((x - 4)/3)$ is $+ \uparrow + x\ y\ 2\ /\ -\ x\ 4\ 3$.



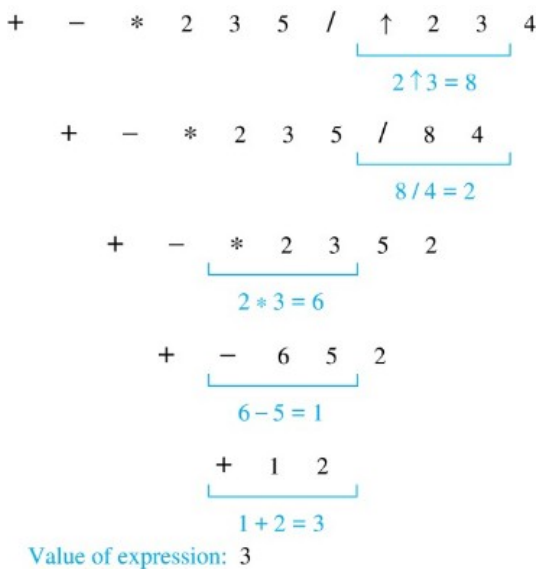






Prefix expressions are evaluated by working from right to left. When we encounter an operator算子, we perform the corresponding operation with the two operands运算数 to the right.

Example: We show the steps used to evaluate a particular prefix expression:

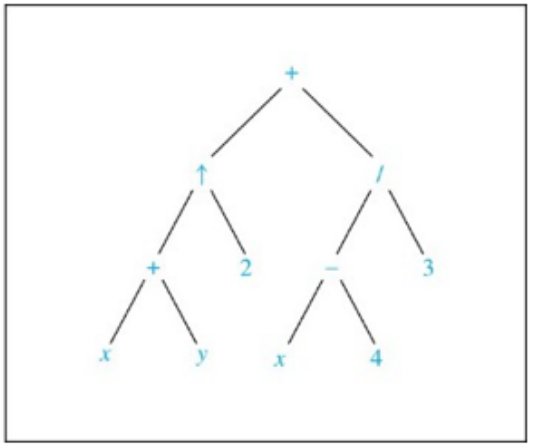


Postfix Notation

We obtain the *postfix form* *后缀符号法*of an expression by traversing its binary trees in postorder. Expressions written in postfix form are said to be in *reverse Polish notation* *逆波兰符号法.*

Parentheses are not needed as the postfix form is unambiguous.

x y + 2 ↑ *x* 4 − 3 / + is the postfix form of ((*x* + *y*) ↑ 2) + ((*x* − 4)/3).



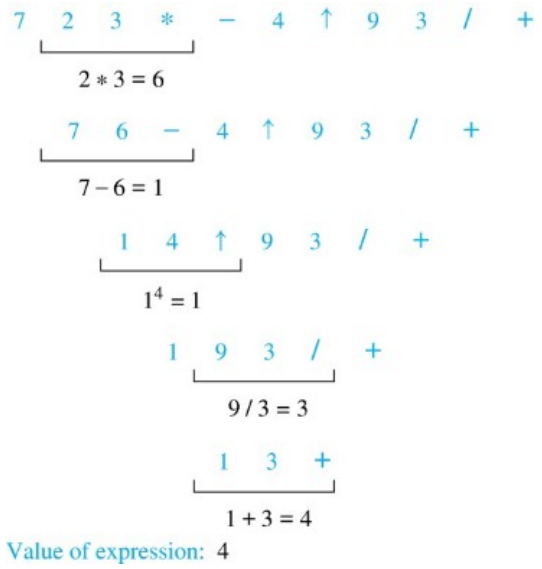
https://blog.csdn.net/Antineutrino/article/details/6763722

Jump to long description

Postfix Notation

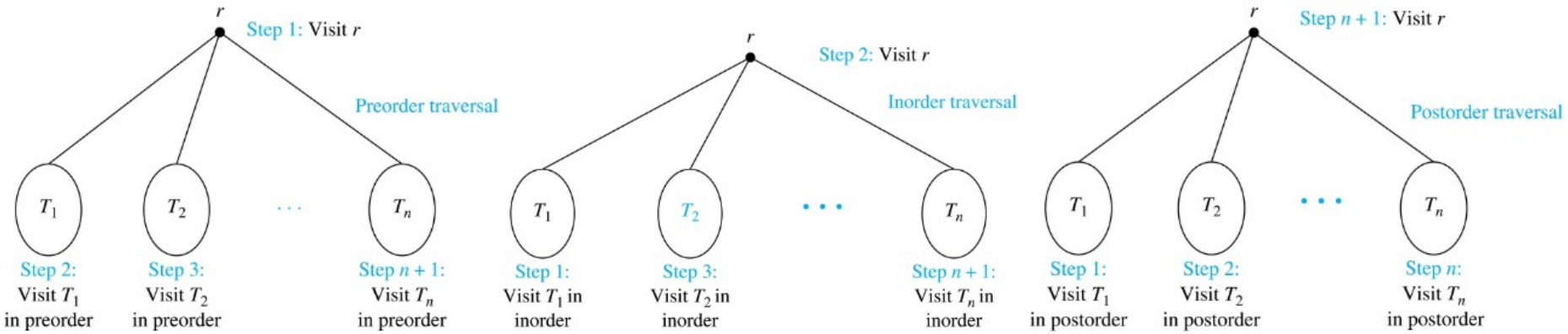
In the postfix form of an expression , a binary operator follows its two operands. So, to evaluate an expression from its postfix form, work from left to right, carrying out operations whenever an operator算子 follows two operands运算数.

Example: We show the steps used to evaluate a particular postfix expression.



Section Summary 2

Traversal Algorithms



Infix Notation (parentheses are needed),

Prefix, and Postfix Notation(Parentheses are not needed)

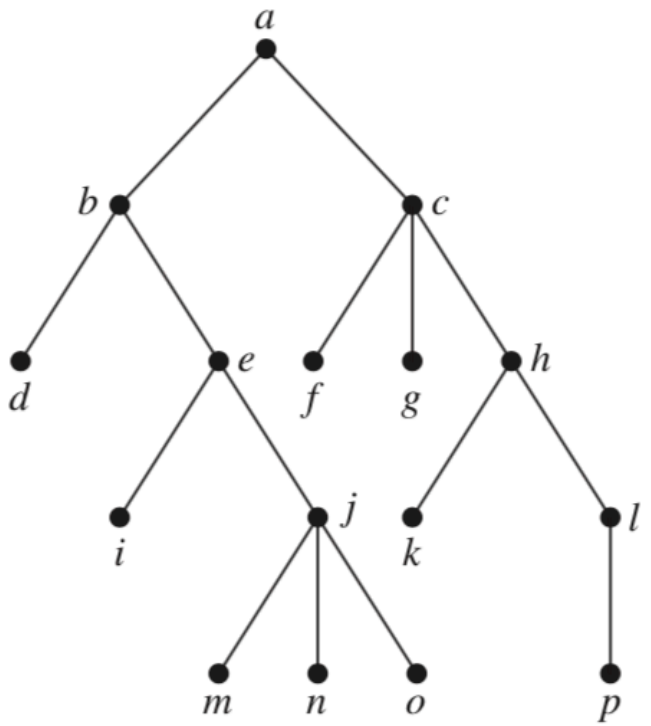
Prefix expressions are evaluated by working from right to left. To evaluate an expression from its postfix form, work from left to right.

In Exercises 7–9 determine the order in which a preorder traversal visits the vertices of the given ordered rooted tree.

11. In which order are the vertices of the ordered rooted tree in Exercise 8 visited using an inorder traversal?

14. In which order are the vertices of the ordered rooted tree in Exercise 8 visited using a postorder traversal?

8.



17. a) Represent the expressions $(x + xy) + (x/y)$ and $x + ((xy + x)/y)$ using binary trees.

Write these expressions in

b) prefix notation.

c) postfix notation.

d) infix notation.

22. Draw the ordered rooted tree corresponding to each of these arithmetic expressions written in prefix notation. Then write each expression using infix notation.

b) $\uparrow + 2\ 3 - 5\ 1$

23. What is the value of each of these prefix expressions?

b) $\uparrow - \ * \ 3 \ 3 \ * \ 4 \ 2 \ 5$

24. What is the value of each of these postfix expressions?

b) $9 \ 3 \ / \ 5 + 7 \ 2 - *$

Chapter Summary

Introduction to Trees

Tree Traversal

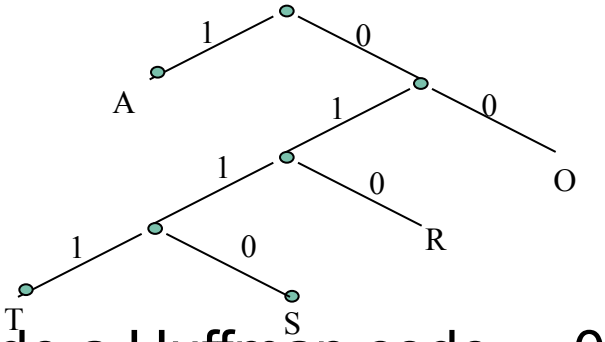
Exercise

Page 820 7 10 13 16 22a 23a 24a

11.2 Applications of Trees

Huffman code 霍夫曼编码uses short bit to represent most frequently used characters, and longer bit strings for the less frequently ones.

Example:



- How to decode a Huffman code 01010111.

  (1) start at root, 0 --> 1 --> 0 move down the tree until a character is reached. The first one is “R”.

  (2) Now look at the remaining string “10111”. Go back to root again. The first bit “1” hits character “A”.

  (3) Now look at the remaining string “0111”. Go back to root again. 0 --> 1 --> 1 --> 1. The character “T” is hit.

  Final result: the Huffman code 01010111 represents the character string “RAT”.

Huffman code

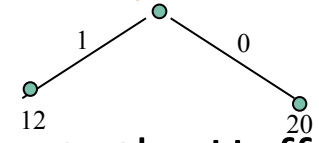
- How to construct an optional Huffman code from a table giving the frequency of occurrence of the characters to be represented?

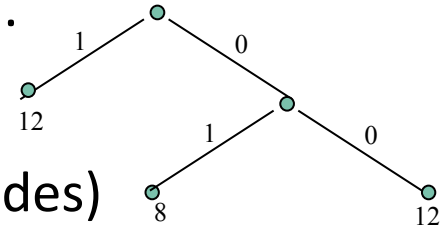
Given character	Frequency
!	2
@	3
#	7
$	8
%	12

- **Algorithm**

1. Replace the smallest two frequencies with their sum 2, 3 --> 2+3.

2. Repeat step (1) until a 2-element sequence is obtained 2, 3, 7, 8, 12 --> 2+3, 7, 8, 12 --> 5+7, 8, 12 --> 8+12, 12 --> 20, 12 .

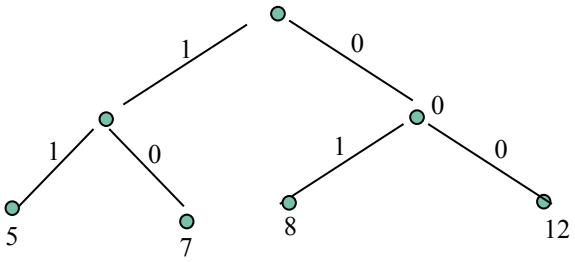




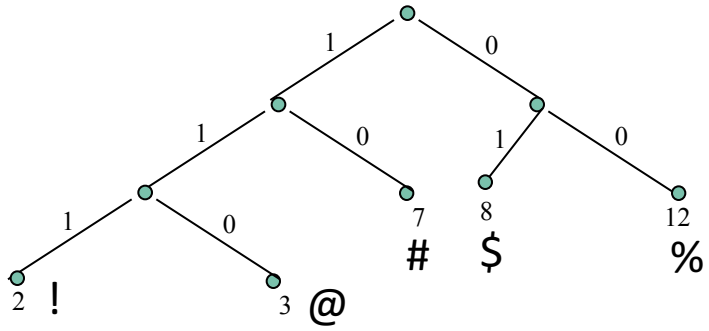
3. Construct the first tree (that represents the Huffman codes) backward beginning with 2-element sequence 20, 12. The element with smallest value is put under the "1" branch of the tree

4. Construct the second tree by decomposing the vertex 20 into 12+8.

5. Repeat this decomposition process until the original sequence of frequency numbers all appear as vertices of the tree.

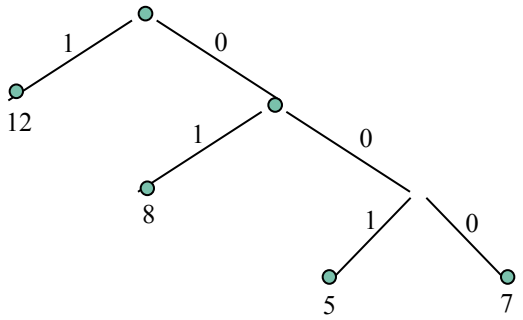


→

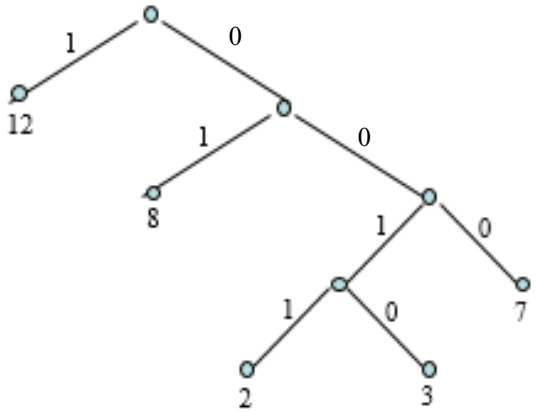


Optimal Huffman tree

Note: at step (5), there are two “12” nodes. Alternative way to do the decomposition:



→



2, 3, 7, 8, 12 --> 2+3, 7, 8, 12 --> 5+7, 8, 12 --> 8+12, 12 --> 20, 12 .

Example 5 Huffman coding

Use Huffman coding to encode the following symbols with the frequencies listed: A: 0.08, B: 0.10, C: 0.12, D: 0.15, E: 0.20, F: 0.35. What is the average number of bits used to encode a character?

Solution: Figure 6 displays the steps used to encode these symbols. The encoding produced encodes A by 111, B by 110, C by 011, D by 010, E by 10, and F by 00. The average number of bits used to encode a symbol using this encoding is

$$3 \cdot 0.08 + 3 \cdot 0.10 + 3 \cdot 0.12 + 3 \cdot 0.15 + 2 \cdot 0.20 + 2 \cdot 0.35 = 2.45.$$

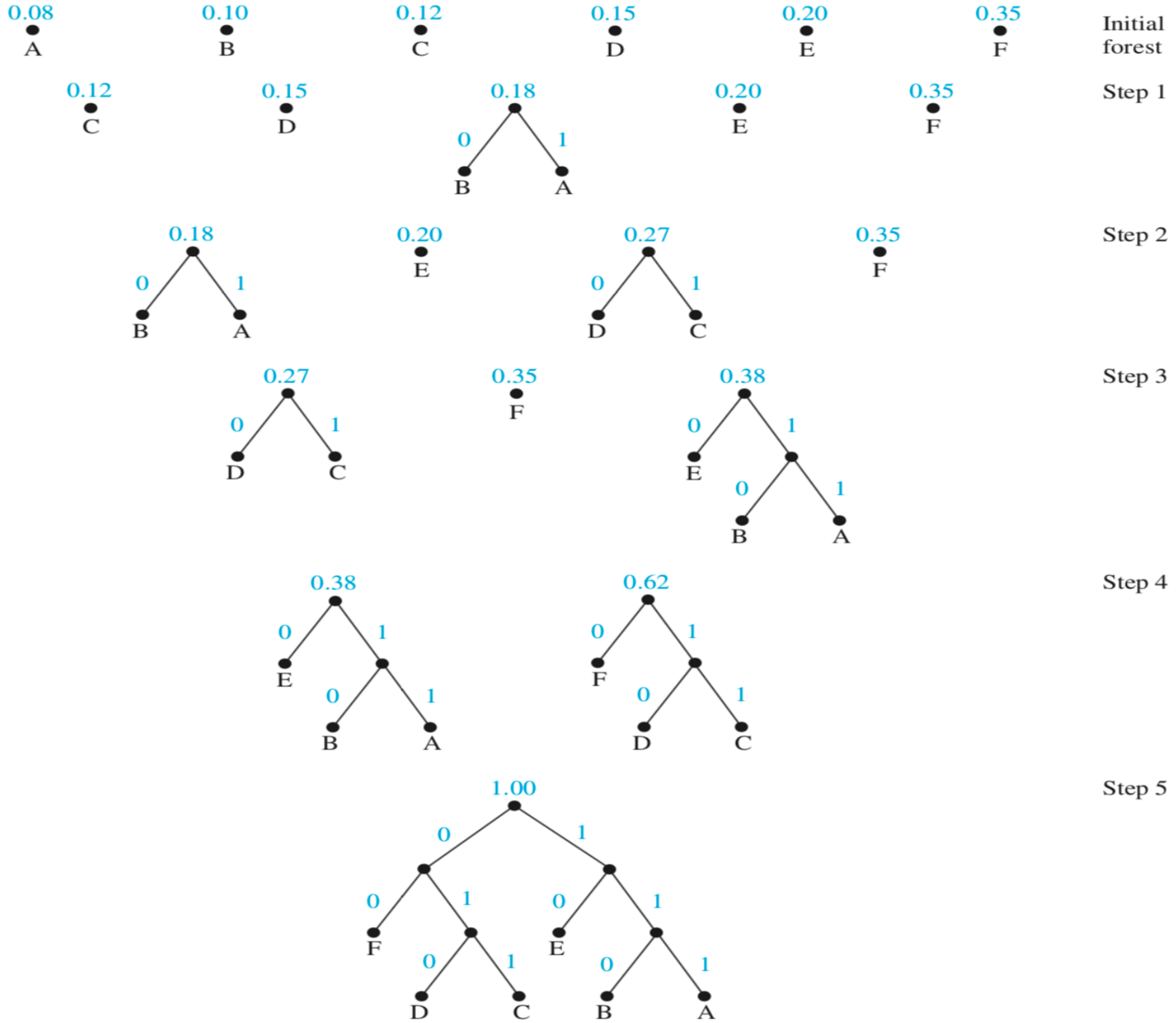


FIGURE 6 **Huffman coding of symbols in Example 4.**

Decision Tree

We display in Figure 4 a decision tree that orders the elements of the list a, b, c.

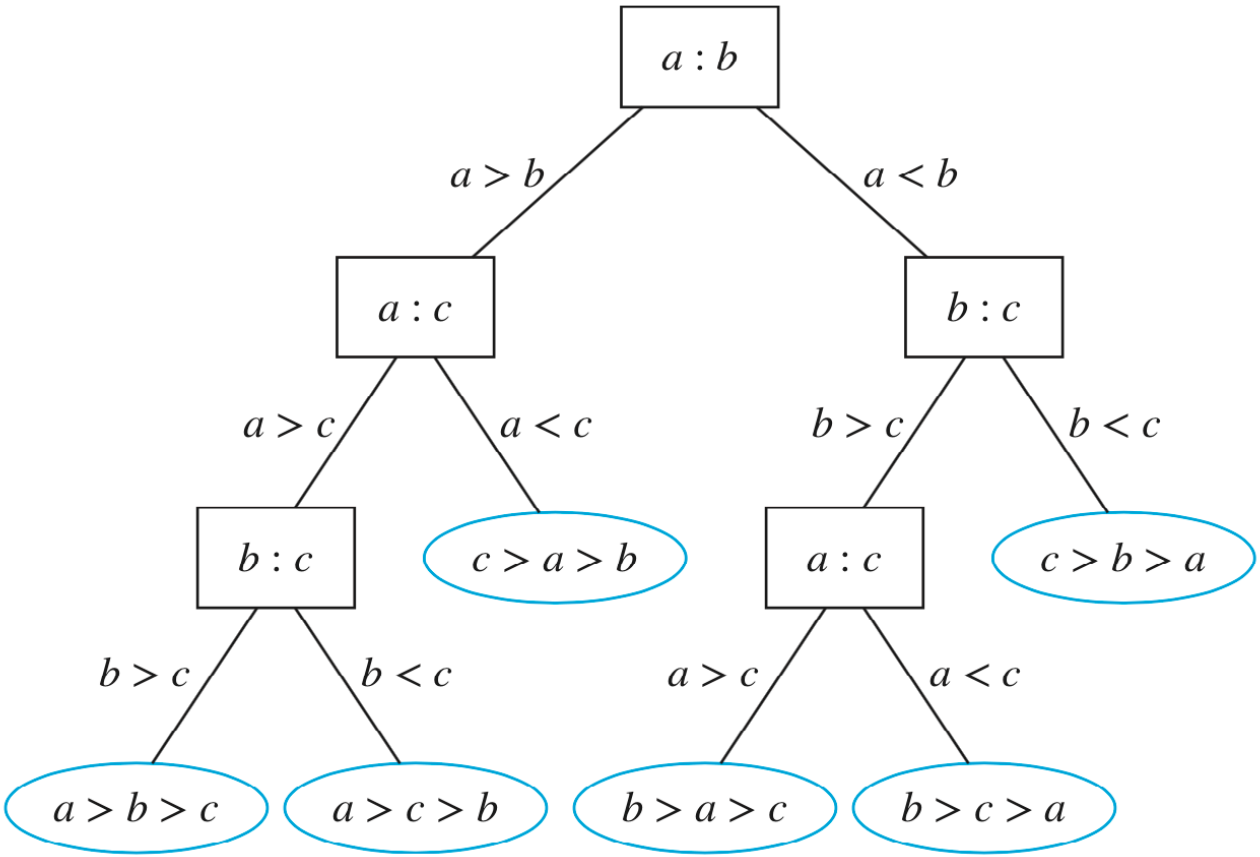


FIGURE 4 **A decision tree for sorting three distinct elements.**

Theorem 5: There are at most m^h leaves in an m-ary tree of height h.
Corollary 1: If an m-ary tree of height h has l leaves, then $h \geq \lceil \log_m l \rceil$.

The complexity of a sort based on binary comparisons is measured in terms of the number of such comparisons used. The largest number of binary comparisons ever needed to sort a list with n elements gives the worst-case performance of the algorithm. The most comparisons used equals the longest path length in the decision tree representing the sorting procedure. In other words, the largest number of comparisons ever needed is equal to the height of the decision tree. Because the height of a binary tree with n! leaves is at least ⌈log2 n!⌉(using Corollary1 in Section11.1), at least ⌈log2 n!⌉ comparisons are needed, as stated in Theorem1.

THEOREM 1 A sorting algorithm based on binary comparisons requires at least $\lceil \log_2 n! \rceil$ comparisons.

We can use Theorem1 to provide a big-Omega estimate for the number of comparisons used by a sorting algorithm based on binary comparison. We need only note that by Exercise 74 in Section 3.2 we know that ⌈logn!⌉ is Θ(nlogn), one of the commonly used reference functions for the computational complexity of algorithms. Corollary 1 is a consequence of this estimate

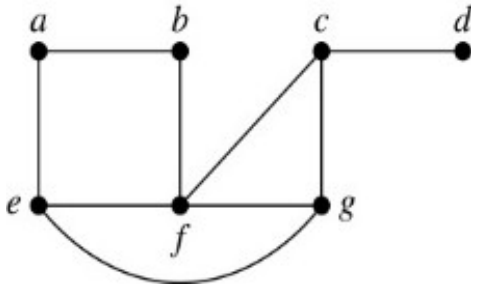
COROLLARY 1 The number of comparisons used by a sorting algorithm to sort n elements based on binary comparisons is $\Omega(n \log n)$.

Spanning Trees

Section 11.4

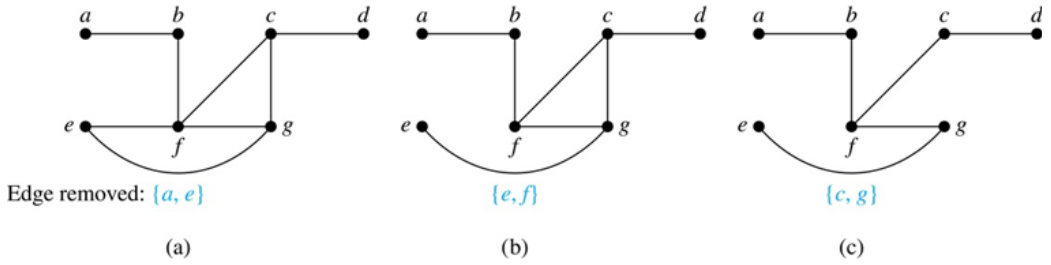
Spanning Trees 1

Definition: Let G be a simple graph. A spanning tree of G is a subgraph of G that is a tree containing every vertex of G.



Example: Find the spanning tree of this simple graph:

Solution: The graph is connected, but is not a tree because it contains simple circuits. Remove the edge $\{a, e\}$. Now one simple circuit is gone, but the remaining subgraph still has a simple circuit. Remove the edge $\{e, f\}$ and then the edge $\{c, g\}$ to produce a simple graph with no simple circuits. It is a spanning tree, because it contains every vertex of the original graph.



11.5 Minimum Spanning Trees

11.5.2 Algorithms for Minimum Spanning Trees

A *minimum spanning tree* in a connected weighted graph is a spanning tree that has the smallest possible sum of weights of its edges.

Algorithm1 Prim's Algorithm

ALGORITHM 1 Prim's Algorithm.

procedure *Prim*(G: weighted connected undirected graph with n vertices)
T := a minimum-weight edge
for $i := 1$ **to** $n - 2$
    e := an edge of minimum weight incident to a vertex in T and not forming a simple circuit in T if added to T
    $T := T$ with e added
return T {T is a minimum spanning tree of G}

Prim's Algorithm to find minimum spanning tree.

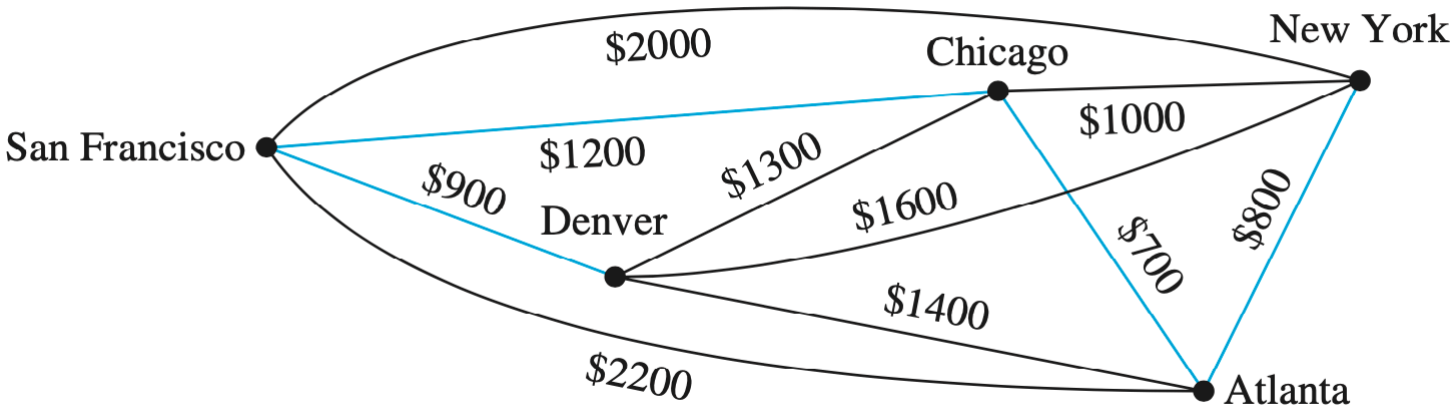
i. Find an edge $e_1 \in E(G)$ with smallest weight and Let $T_1 = e_1$.

ii. Among all edges adjacent with T_1, find e_2 with smallest weight and do not contain any vertex already visited, and Let $T_2 = T_1 + e_2$.

iii. Repeat i and ii until all vertices are included.

Example 1

Use Prim's algorithm to design a minimum-cost communications network connecting all the computers represented by the graph in Figure 1.



Choice	Edge	Cost
1	{Chicago, Atlanta}	$ 700
2	{Atlanta, New York}	$ 800
3	{Chicago, San Francisco}	$1200
4	{San Francisco, Denver}	$ 900
	Total:	$3600

FIGURE 2 **A minimum spanning tree for the weighted graph in Figure 1.**

Example 2

Use Prim's algorithm to find a minimum spanning tree in the graph shown in Figure 3.

Solution: A minimum spanning tree constructed using Prim's algorithm is shown in Figure 4. The successive edges chosen are displayed.

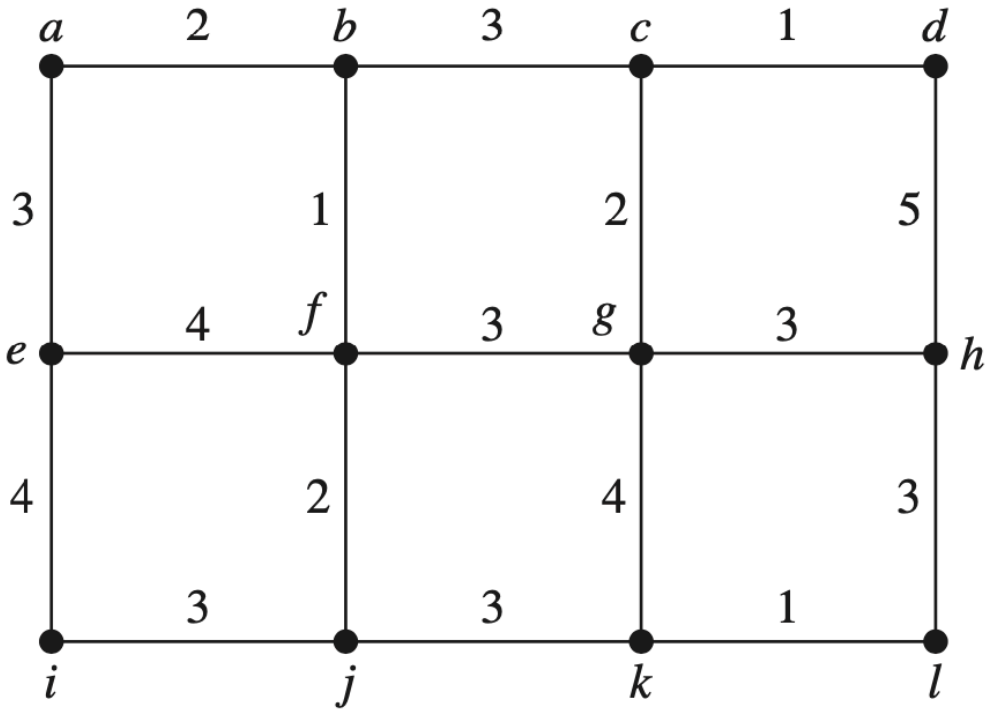
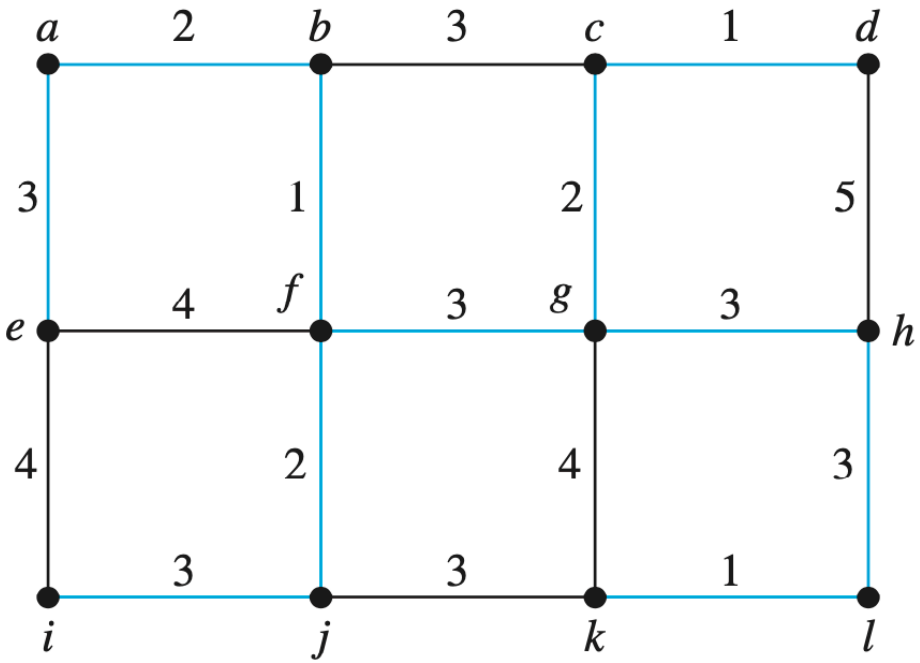


FIGURE 3 **A weighted graph.**



(a)

Choice	Edge	Weight
1	$\{b, f\}$	1
2	$\{a, b\}$	2
3	$\{f, j\}$	2
4	$\{a, e\}$	3
5	$\{i, j\}$	3
6	$\{f, g\}$	3
7	$\{c, g\}$	2
8	$\{c, d\}$	1
9	$\{g, h\}$	3
10	$\{h, l\}$	3
11	$\{k, l\}$	1
	Total:	24

(b)

FIGURE 4 **A minimum spanning tree produced using Prim's algorithm.**

Algorithm 2 Kruskal's Algorithm

ALGORITHM 2 Kruskal's Algorithm.

procedure *Kruskal*(G: weighted connected undirected graph with n vertices)
T := empty graph
for $i := 1$ **to** $n - 1$
    e := any edge in G with smallest weight that does not form a simple circuit
        when added to T
    $T := T$ with e added
return T {T is a minimum spanning tree of G}

Kruskal's Algorithm to find minimum spanning tree.

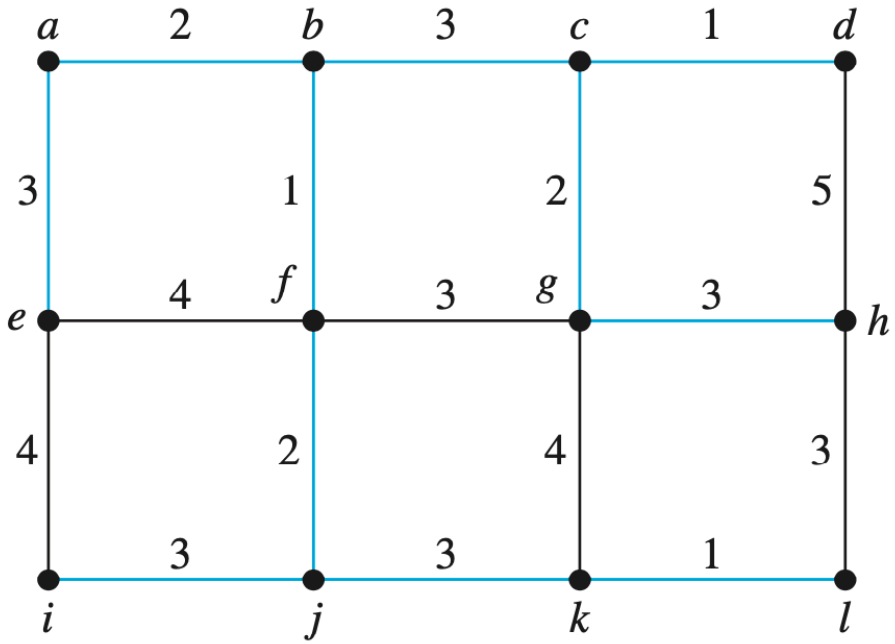
i. Find an edge e of G such that e has a smallest weight and do not form a simple circuit.

ii. Repeat i to add edges unitil every vertex been included.

Example 3

Use Kruskal's algorithm to find a minimum spanning tree in the weighted graph shown in Figure 3.

Solution: A minimum spanning tree and the choices of edges at each stage of Kruskal's algorithm are shown in Figure 5.



(a)

Choice	Edge	Weight
1	$\{c, d\}$	1
2	$\{k, l\}$	1
3	$\{b, f\}$	1
4	$\{c, g\}$	2
5	$\{a, b\}$	2
6	$\{f, j\}$	2
7	$\{b, c\}$	3
8	$\{j, k\}$	3
9	$\{g, h\}$	3
10	$\{i, j\}$	3
11	$\{a, e\}$	3
	Total:	24

(b)

FIGURE 5 A minimum spanning tree produced by Kruskal's algorithm.